DSP Integrated Circuits

Chapter 6: DSP H/W Design

Prof. Jaeseok Kim

Communication SoC Design Lab.

Yonsei University

Contents

- H/W Key Components for DSP design
- Analog I/O

H/W Key Components

H/W Key Components (1)

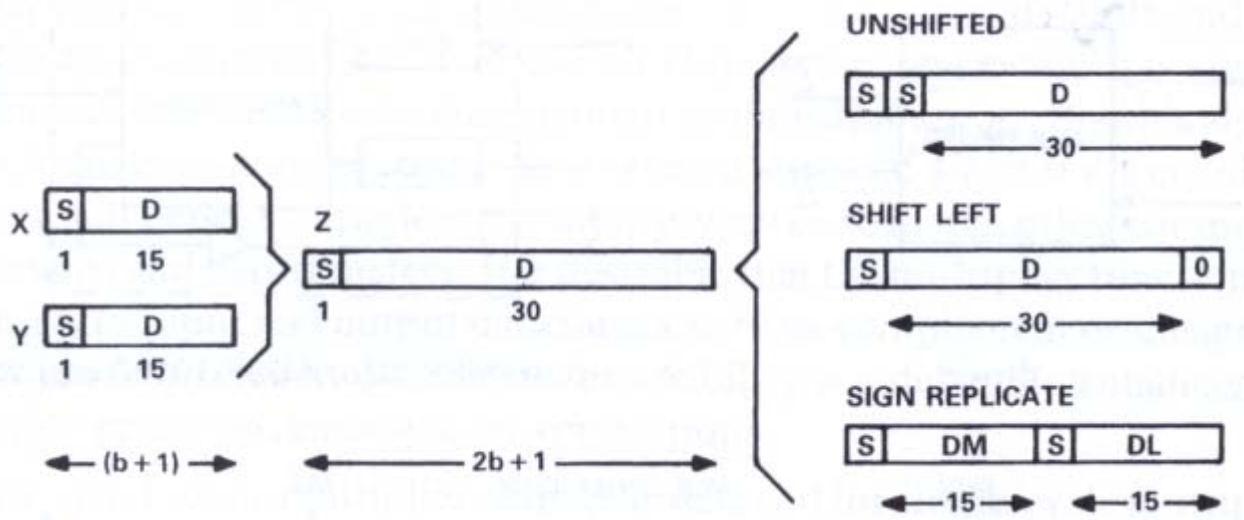
- Multiplier
- Multiplier-Accumulator (MAC)
- Arithmetic Logic Unit (ALU)
- Shifter
- Address Generator (AG)
- Program Sequencer (PS)
- Memory Architecture

H/W Key Components (2)

- Multiplier (1)
  - Desirable Features
    - Speed : comparable to RAM access time; fully parallel operation
    - Input Formats : 2's complement or unsigned, fractional or integer
    - Mixed mode : Signed or unsigned—a desirable enhancement
    - Output Formats : Right-shifted ("Integer") or left-shifted ("Fractional")
    - Extended or double-precision option
    - Fixed point vs. floating-point tradeoffs
    - Single-cycle multiply of two inputs X and Y; bussing, pipelining issues
    - Input and output registers configurable for latching (pipelined operation) or transparence (minimum latency)

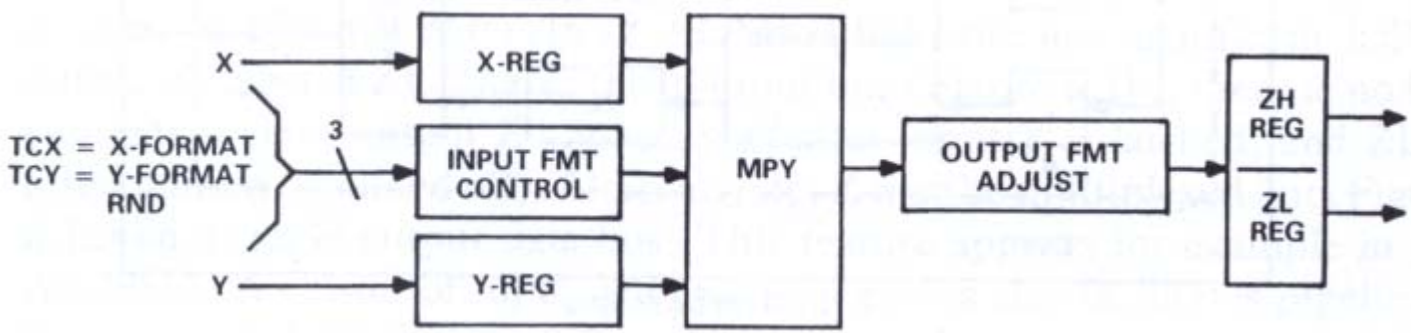
H/W Key Components (3)

- Multiplier (2)
  - Bit and Word Formats
    1) The Extra Bit in 2's complement multiplication



H/W Key Components (4)

- Multiplier (3)
  - Bit and Word Formats
    - 2) Sign control and mixed mode



X	Y	Result
Signed	Signed	Signed
Unsigned	Unsigned	Unsigned
Signed	Unsigned	Signed (mixed-mode)
Unsigned	Signed	Signed (mixed-mode)

H/W Key Components (5)

- Multiplier (4)
  - Bit and Word Formats
    - 3) Rounding Issues

	MSH	LSH	
	10011011	10101010	Full-width output, LSH > FS/2
	+	10000000	Augment LSH, then truncate →
=	10011100		Rounded output incremented
	10011011	00101010	Full-width output, LSH < FS/2
	+	10000000	Augment LSH, then truncate →
=	10011011		Rounded output unchanged

  - Speed, accuracy, cost tradeoffs
  - Pipelined multipliers
    - Without pipelining
      - One-cycle output delay, Low frequency
    - With pipelining
      - Multiple output delay, High frequency

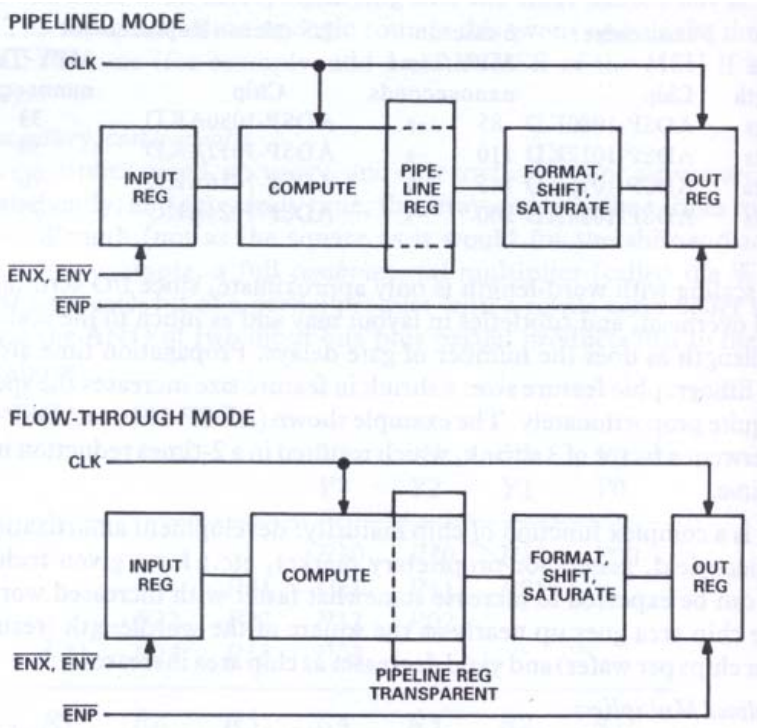
H/W Key Components (6)

Multiplier-Accumulator (MAC) (1)

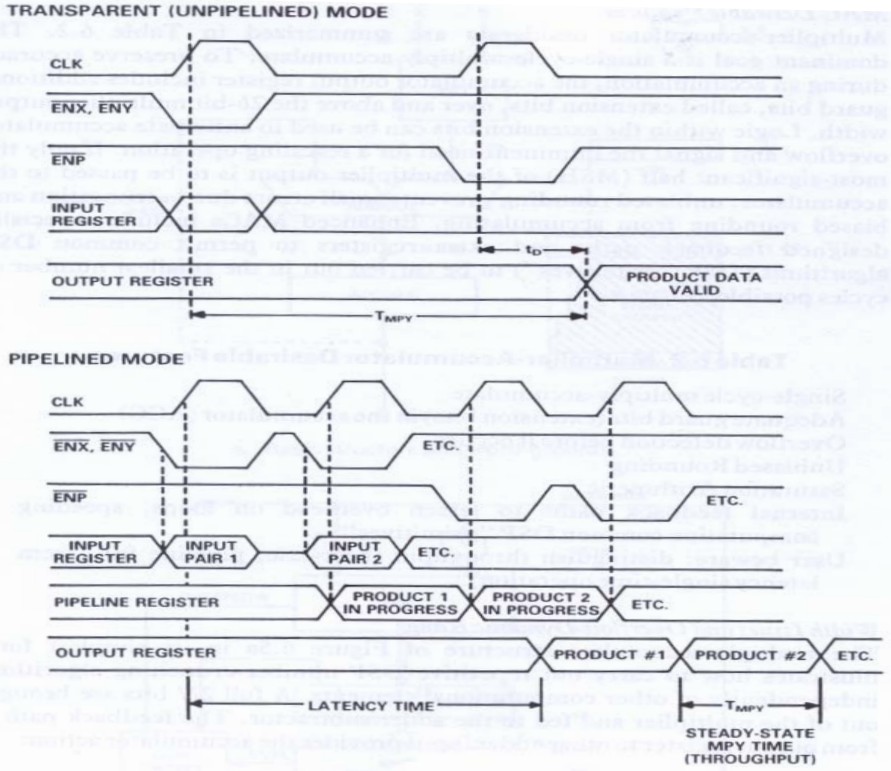
- Desirable Features
  - Single-cycle multiply-accumulate
  - Adequate guard bits (extension bits) in the accumulator (ACC)
  - Overflow detection before it occurs
  - Unbiased Rounding
  - Saturation Arithmetic
  - Internal feedback paths to lessen overhead on loops, speeding computation common DSP “primitives”
  - User beware : distinguish throughput (best-case, pipeline full) from latency single-time operation)

H/W Key Components (7)

- MAC (2)
  - Pipelined multiplier with transparent option



a. Structure.



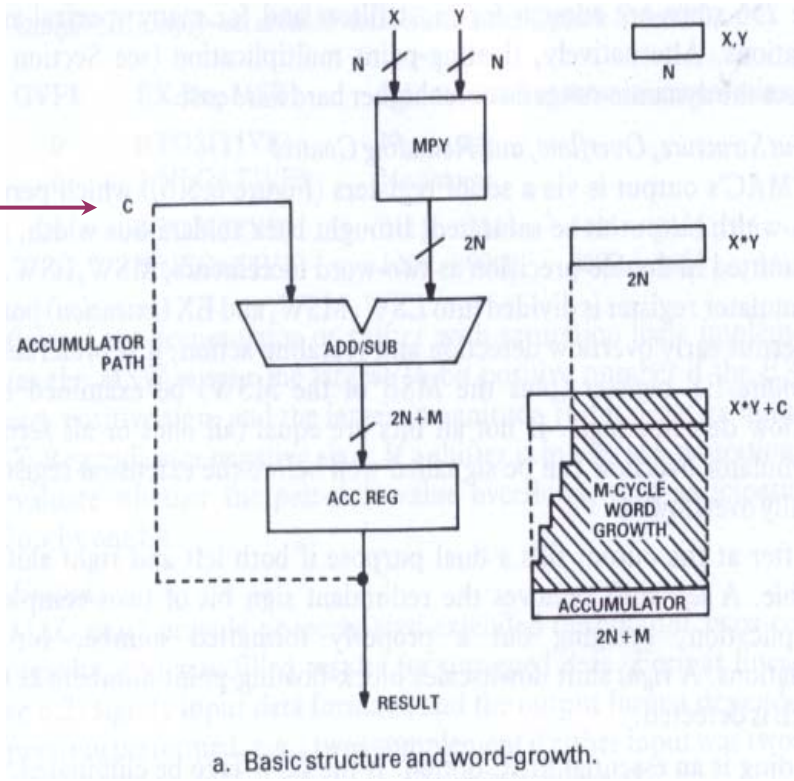
b. Data timing in unpipelined (or transparent) and pipelined modes.

H/W Key Components (8)

- MAC (3)
  - Width Issues and overflow dynamic range

The Feedback path C provides the accumulator action :

$$R(n+1) = X(n+1) * Y(n+1) \pm R(n)$$

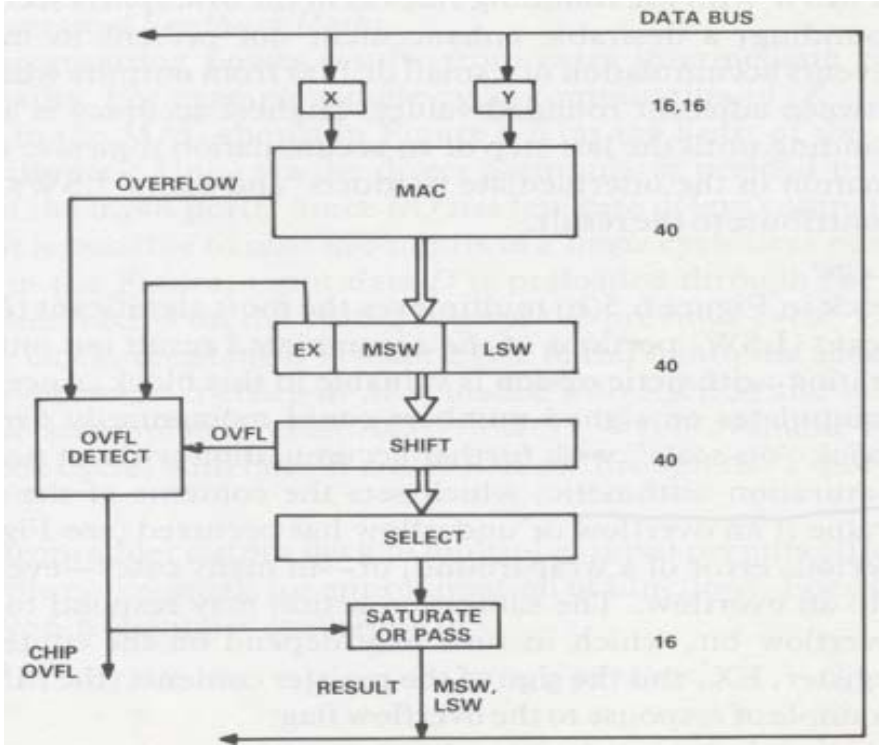


a. Basic structure and word-growth.

H/W Key Components (9)

MAC (4)

- Output structure, overflow, rounding control and saturation Logic
  - The extra-width output to be saturated, brought back to data-bus width, and/or transmitted in MSW and LSW
  - Rounding is essential MAC option, but highest accuracy is attained by avoiding rounding

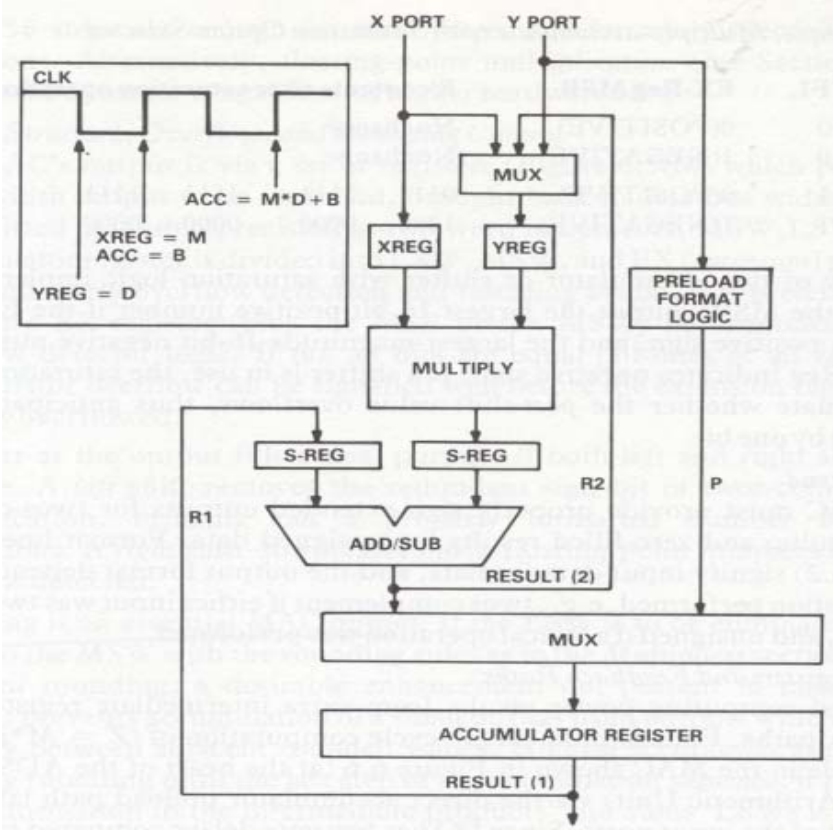


b. Elements of the enhanced MAC structure of the ADSP-1101 Integer Arithmetic Unit.

* MSW (Most-Significant Word)
* LSW (Lest-Significant Word)

H/W Key Components (10)

- MAC (5)
  - Extra registers and feedback paths
    - Expanded computing power results from extra intermediate registers and feed back paths



H/W Key Components (11)

Arithmetic Logic Unit (ALU) (1)

- Desirable Features
  - Adequate accuracy and chainability for high-precision computation
  - Arithmetic Functions : add, subtract, add with carry, subtract with borrow, absolute value
  - Logic flags : zero, equal, less than, greater than, carry ...
  - Divide and (less important) multiply capability
  - Efficient data movement and computation :
    - Data moves plus arithmetic in same cycle
    - Two operands loadable into the ALU on each cycle
  - Adequate Register Files :
    - Dual-set for context switching (for storing data during interrupts)

      or: Register File for fast access and storage of intermediate results
  - Conditional operations, e.g., add + (Shift if Flag Set) are an asset

H/W Key Components (12)

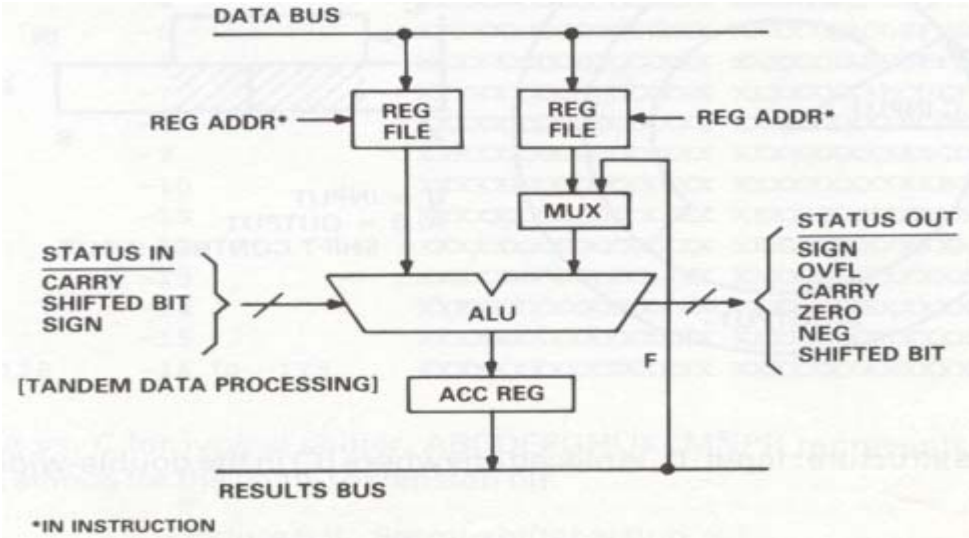
- ALU (2)
  - Desirable Features
    - Conditional operations, e.g., add + (Shift if Flag Set) are an asset
    - Adequate Feedback pathways : Output to input (accumulate); and, if linked to shifter, ALU inputs to shifter or shifter inputs to ALU
    - Pipeline or transparent flow through option on input registers
    - Close association with Shifter for scaling operations common in DSP
    - Double-precision capability with minimum lost time

H/W Key Components (13)

- ALU (3)
  - ALU Functions and Data Formats
    - Condition Flags
      - As with a general-purpose ALU, the DSP ALU must signal status information on the computation just performed
    - Rounding
    - Division Capability
      - Division occurs in many DSP systems, e.g., predictive coding of speech
      - Since division is not among basic MPY or MAC operations, it is a key LAU primitive operation
    - Saturation Logic
    - Arithmetic Plus Conditional Shift
      - ALU can conditionally shift its result up or down during the same cycle, depending on the state of an input flag

H/W Key Components (14)

- ALU (4)
  - ALU Architecture Issues
    - Word Width and Easy Paralleling
    - Single Cycle Data-Moves-plus-Arithmetic
    - Adequate Register Set



    - Flexible Internal Data Paths and I/O Pathways

H/W Key Components (15)

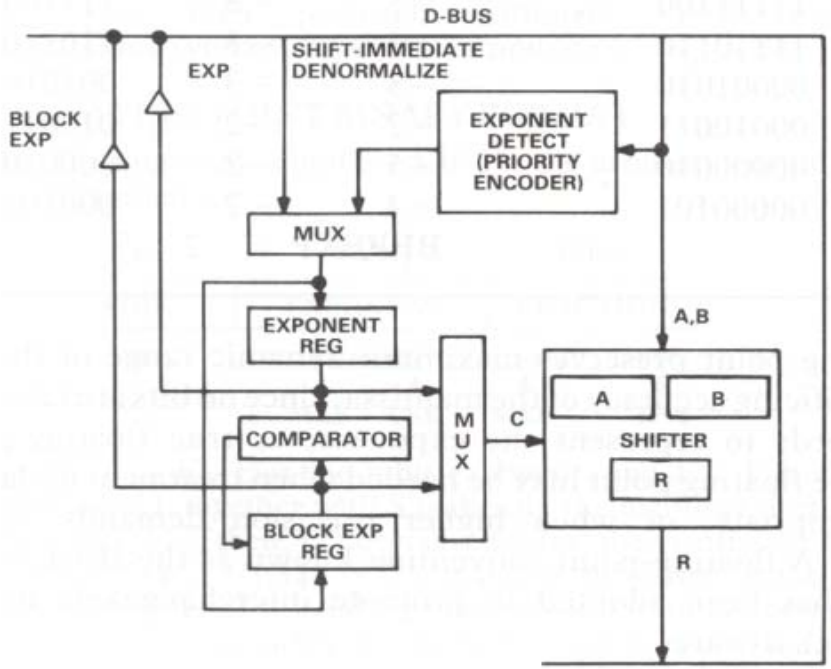
Shifter (1)

- Shifter Definition and Functions
  - A shifter scales numbers to prevent underflows and overflows and performs conversions between fixed-and floating point
  - Unlike the bit-per-cycle shift of general-purpose microprocessors, the DSP shifter must be capable of shifting a word by many bits in a single machine-cycle to preserve single-cycle computation speed
- Basic Shifter Operations
  - N-bit arithmetic and logical shifts (shift-immediate)
  - Word rotations and word merges
  - Bit-test/bit-set/bit-clear

H/W Key Components (16)

Shifter (2)

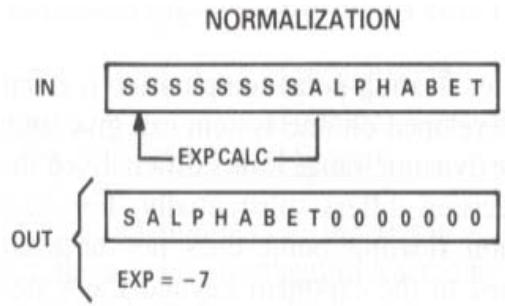
- Shifter Scaling Functions
  - Enhancements for Scaling : provide low-overhead exponent handling
    - Exponent detector : Uses a priority encoder to translate the number of identical leading bits into a binary number representing the number's base-2 exponent



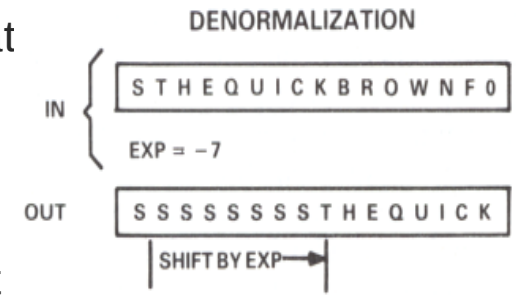
a. Block diagram.

H/W Key Components (17)

- Shifter (3)
  - Shifter Scaling Functions
    - Enhancements for Scaling : provide low-overhead exponent handling
      - Normalization
        : Fixed-to-floating point conversion,
        generating a mantissa and an exponent



      - Denormalization : The inverse of normalizat



      - Block Floating-Point : derives the exponent
        of the largest member of a data array
        - Block floating-point exponents are determined by exponent-detection logic

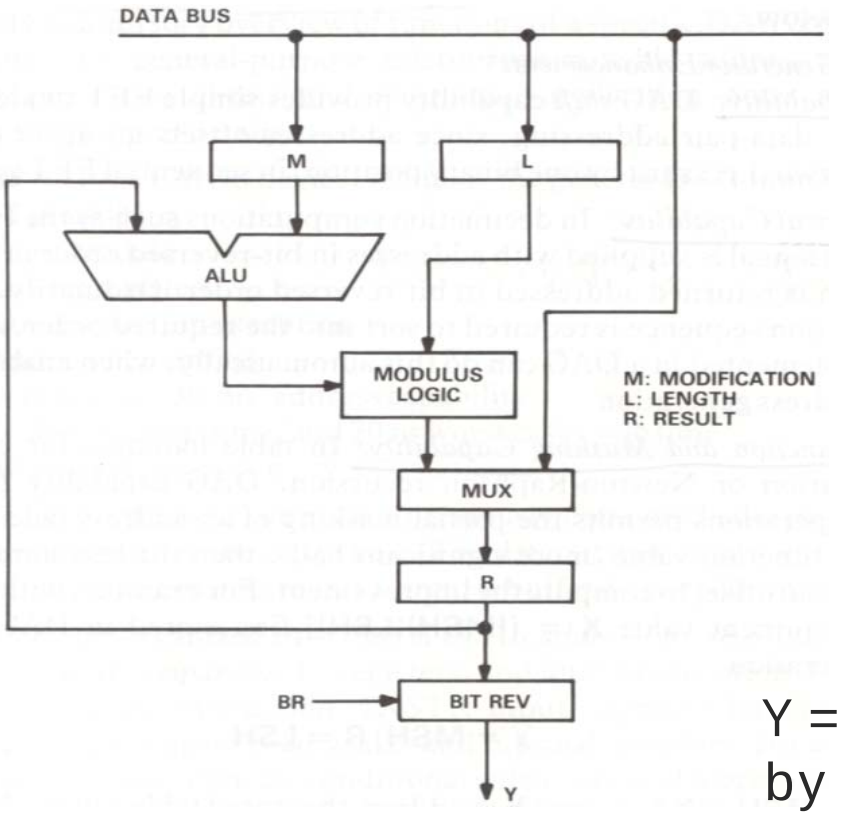
H/W Key Components (18)

Address Generator (AG) (1)

- Desirable Features
  - Send a precomputed address to data memory
    - Data-array scanning : An AG with its own ALU makes possible it
  - Modify the address by a computed offset value
  - Perform logical (AND, OR, XOR) operations and shifts
  - Compare a pointer to a preset value (circular buffer)
  - Reset pointer to buffer origin when pointer = preset
  - Reset to origin when pointer $\geq$ or $\leq$ preset
  - Reverse the order of the bits

H/W Key Components (19)

- AG (2)
  - General scheme of Data Address Generator (DAG)



Y = R; modify R contents
by a computed offset value

H/W Key Components (20)

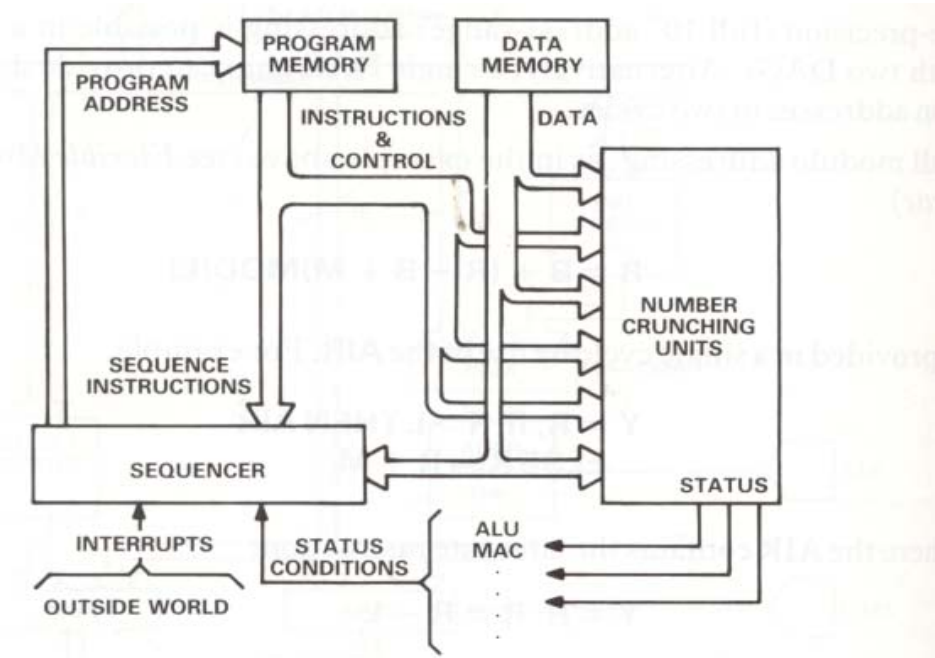
- AG (3)
  - Address Generator Enhancements
    - Shift Capability
      - Provides simple FFT twiddle-table or butterfly data-pair addressing
    - Bit-Reversal Capability
      - Either the input is supplied with addresses in bit-reversed order or the output spectrum is returned addressed in bit-reversed order
    - Logic Function and Masking Capability
      - In table lookups, DAG capability to perform logical operations permits the partial masking of an address field to address a stored function value (more significant half)
    - Word-Slice Address Generator
      - Replaces AGs implemented with several Bit-Slice ALU
      - Programmable, and more flexible solution than fixed-purpose AG chips
    - Flexible Modulus Arithmetic

      $R = B + (R - B + M) \text{ MOD } (L)$

R : the DAG address-register content
B : a base address
M : a computed offset increment
L : the length of the buffer (modulus)

H/W Key Components (21)

- Program Sequencer (PS) (1)
  - Sequencer Definition
    - Generates the instruction addresses which determine program flow



a. Instruction sequencer and its relationship to other DSP system components.

H/W Key Components (22)

- PS (2)
  - Sequencer Functions
    - Handle normal program flow, incrementing the program counter by 1 each cycle
    - Keep track of subroutine addressing, and manage the return-address stack
    - Manage loops in an overhead-free manner, controlled by on-chip loop counters
    - Jump to appropriate exception-handler routines when a data overflow is detected or data rescaling is required
    - Service interrupts from external I/O devices, jumping to appropriate interrupt service routines and returning to the previous task when done

H/W Key Components (23)

- PS (3)
  - Desirable Features : includes the following components
    - Program Counter (PC)
    - Conditional Logic Tester
    - Stack for :
      - Subroutine and Interrupt return addresses
      - Counter values
      - Jump addresses
    - Loop Counters
  - Basic Sequencer Requirements
    - Adequate Stack Size
    - Branch Conditions and Interrupt Prioritization
    - Sequencer Pipelining Is Not Recommended
    - Forcing a Direct Instruction Address
  - Desirable Sequencer Enhancements
    - Stack Boundary Settability
    - Zero-Overhead Conditional Branch Capability

H/W Key Components (24)

Memory Architecture (1)

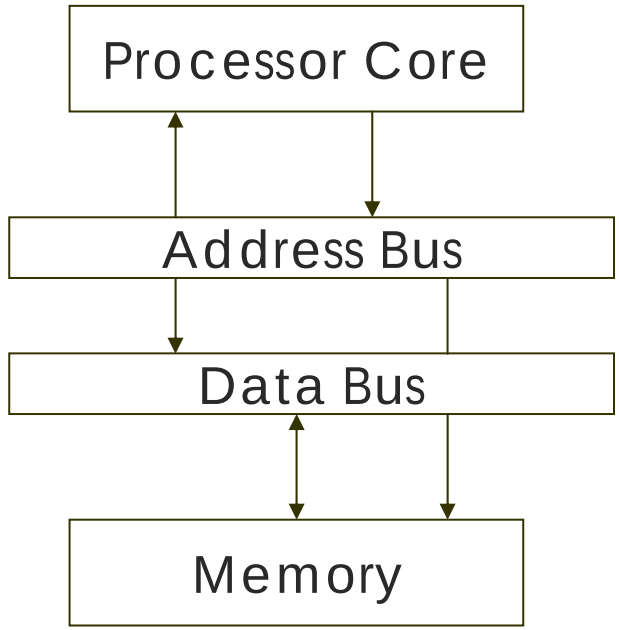
- Desirable Features DSP Memory
  - Separate memory chips to be addressed by DSP components must match their speed
  - Dual-ported memory has two separate ports, controls, address, and I/O lines, to permit independent accessing
  - Register files are another way of increasing memory productively without decreasing speed or adding complexity
  - FIFO registers act as pipelines to match slower memory to faster processors
  - Cache memory : the most recently used instructions remain resident
  - Address generators lessen the demands on memory

$$t(\text{data}) = t(\text{addr. gen.}) + t(\text{data fetch})$$

H/W Key Components (25)

Memory Architecture (2)

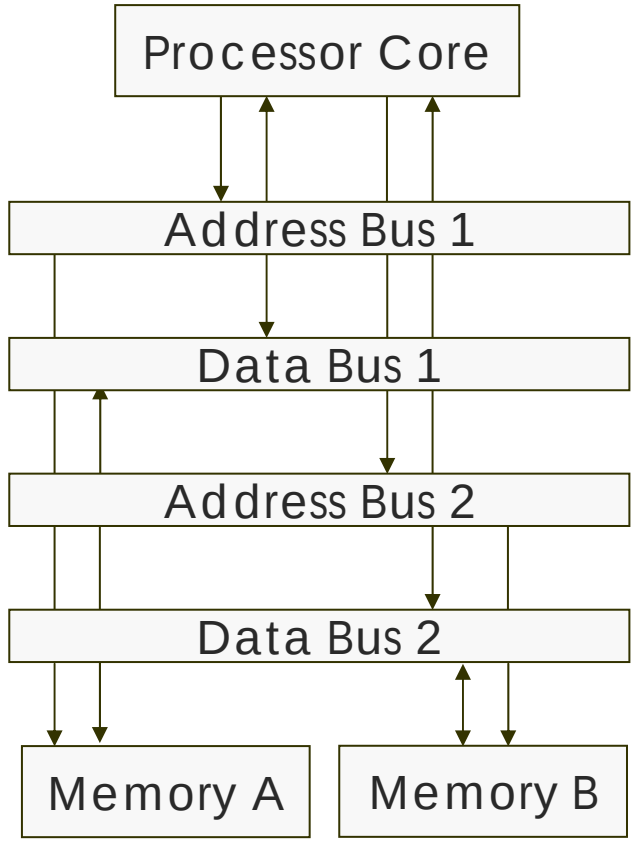
- Von Neumann architecture
  - A single set of address and data lines
    - Both program instructions and data are stored in the single memory
  - Common among non-DSP processors
  - Strength
    - Low-cost
  - Weakness
    - The processor can make one access to memory during each instruction cycle



H/W Key Components (26)

Memory Architecture (3)

- Harvard architecture
  - Tow independent memory banks and two independent sets of buses (address and data)
    - Original
      - One bank for inst. One bank for data
    - Modified
      - One bank for inst. And data, one bank for data
    - Two memory accesses per instruction cycle
  - More powerful derivatives
    - One program bank and two data banks
    - A shortage by one bank can be overcome by module addressing

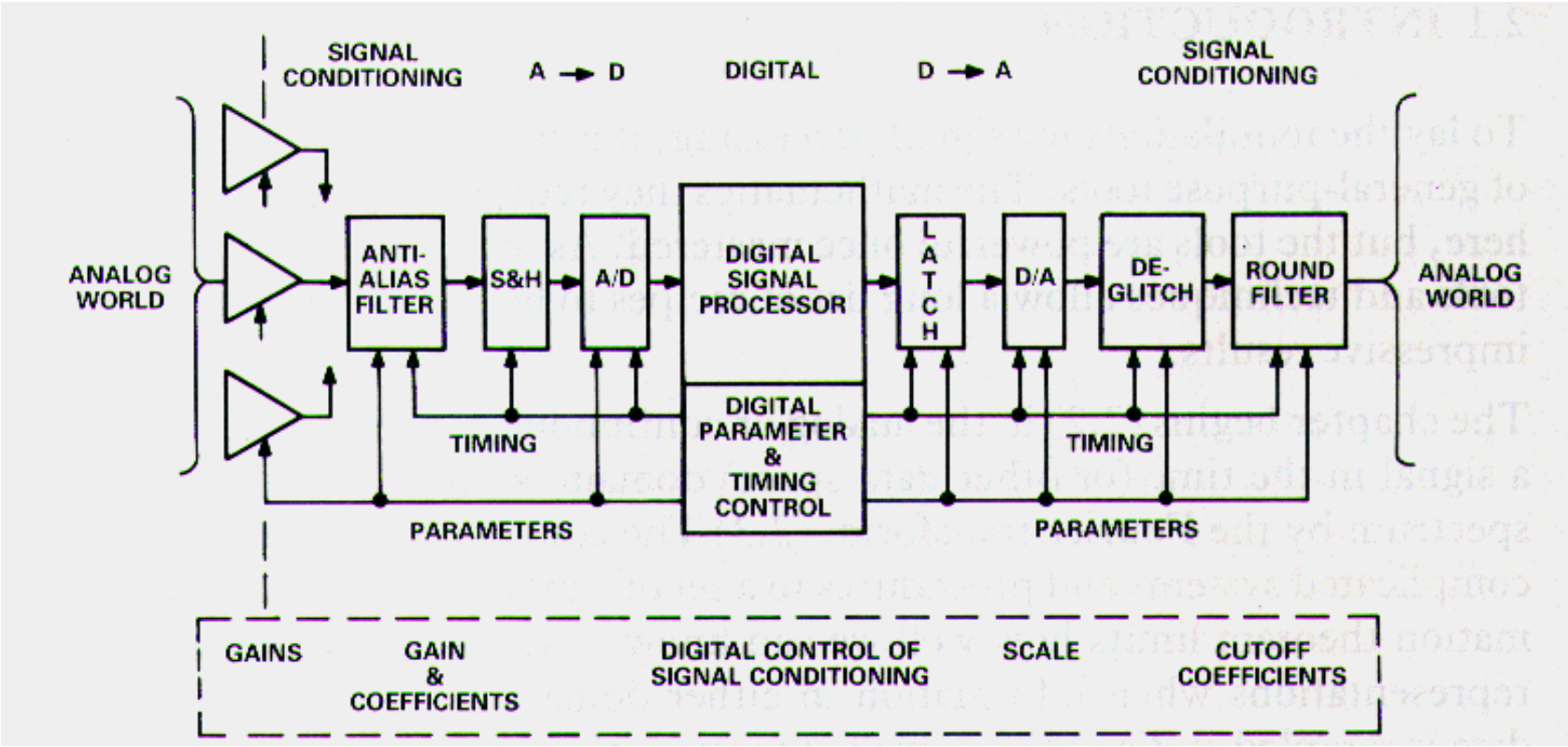


Analog I/O

Analog I/O

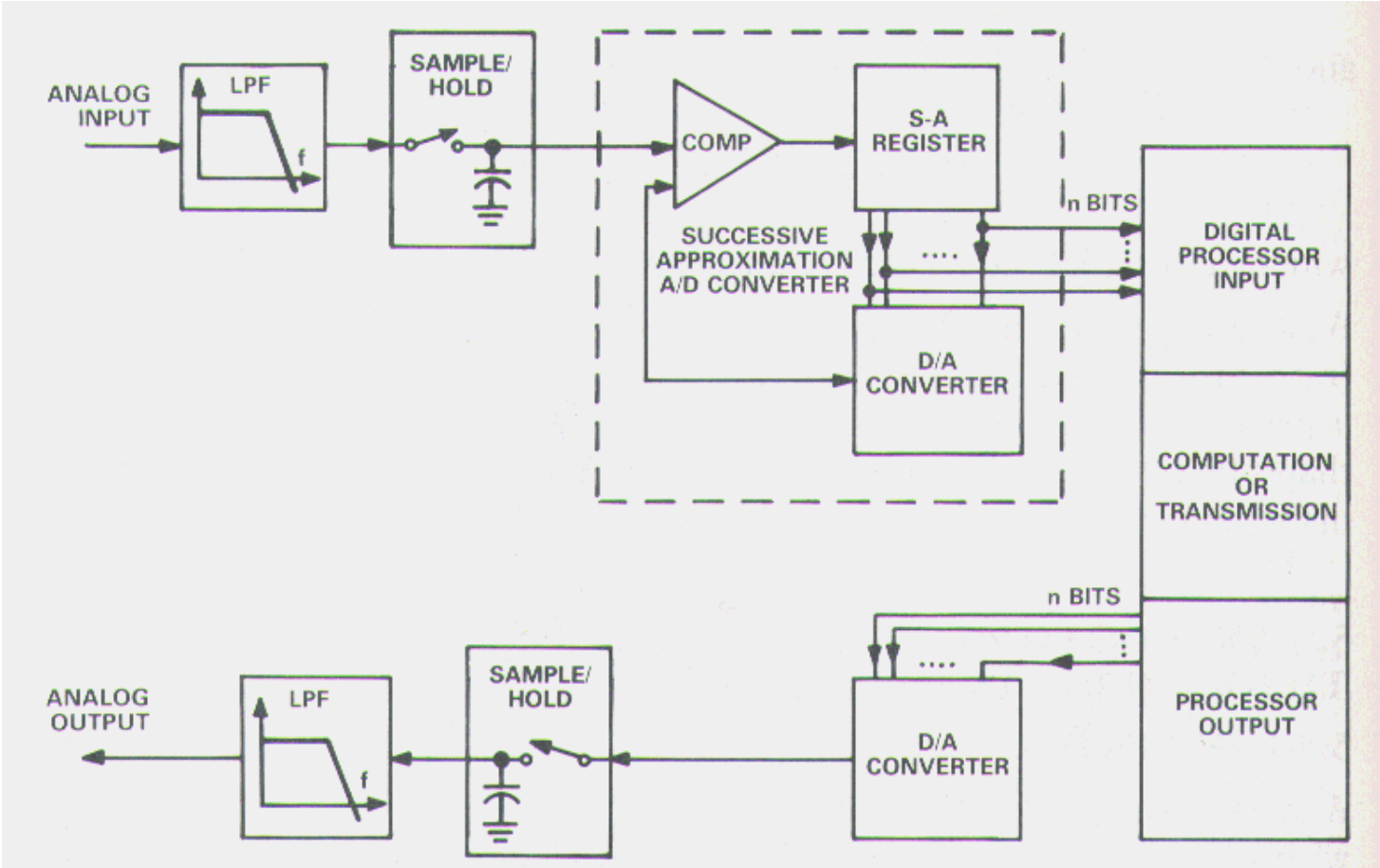
- Analog-to-Digital Conversion
- Automatic Gain Control (AGC)
- Digital-to-Analog Conversion

Analog I/O in DSP System



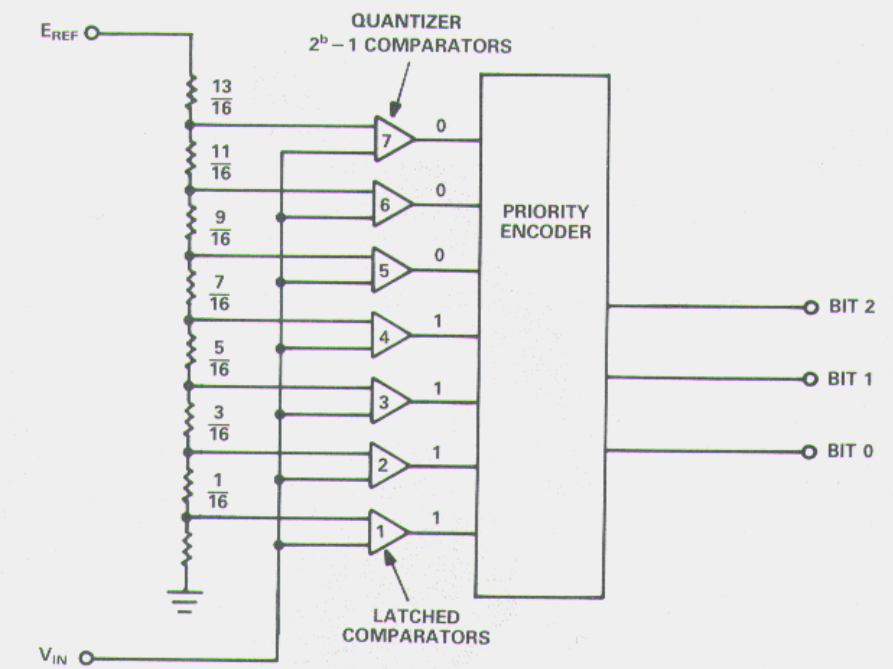
Analog-to-Digital Converter

- Successive Approximation A/D Converter



Analog-to-Digital Converter

- Flash A/D Converter



QUANTUM LEVEL TO BINARY ENCODING

LEVEL	0	1	2	3	4	5	6	7
BIT 2	0	0	0	0	1	1	1	1
BIT 1	0	0	1	1	0	0	1	1
BIT 0	0	1	0	1	0	1	0	1

Comparison

Type	Needs S/H?	Cycles /conversion	Advantages	Example
Successive Approximation	Yes	b (for b-bits output)	cheap	8-bit 500KHz 16-bit 400KHz
Flash	No	1	fast	6-bit 400MHz 10-bit 75MHz

Digital-to-Analog Converter

- Voltage-scaling D/A Converter

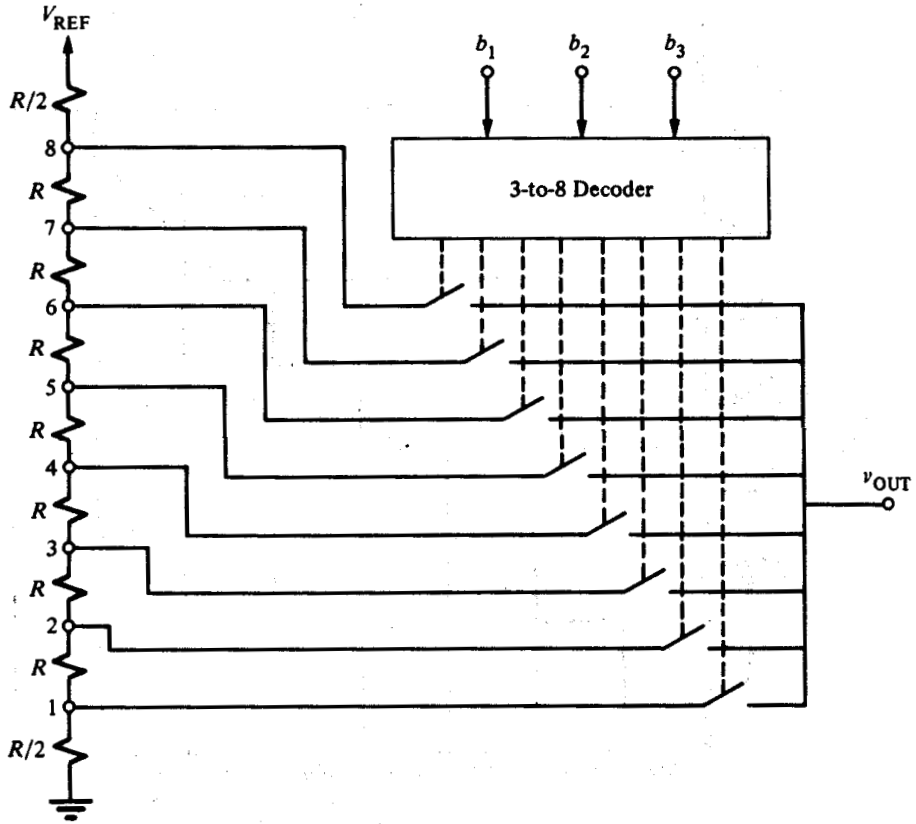
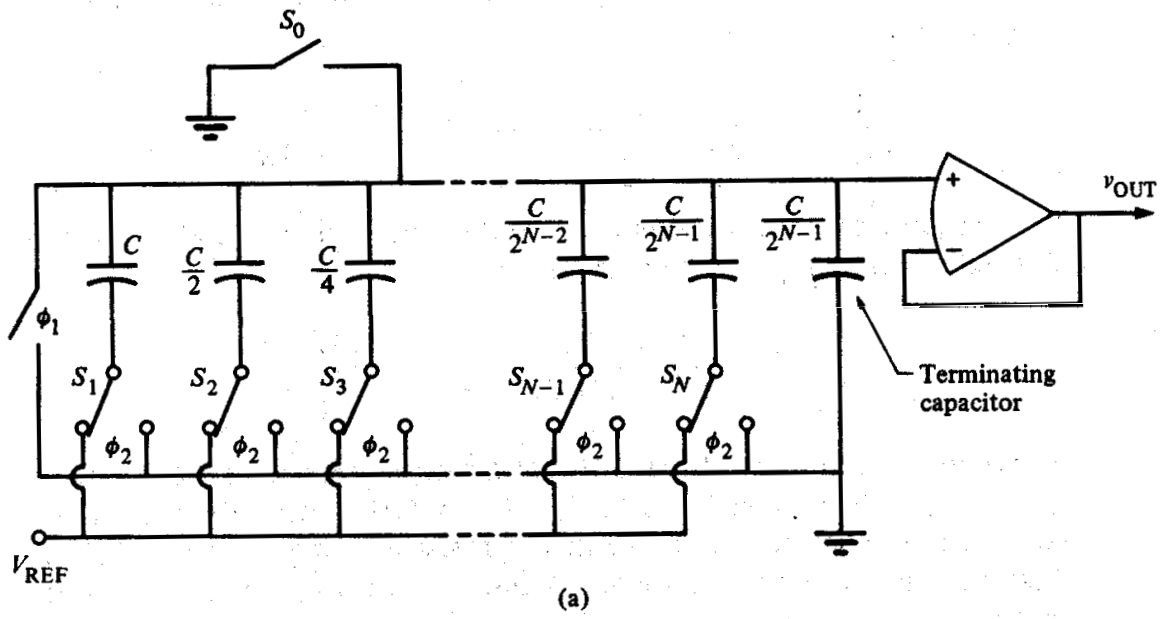


Figure 10.2-2 Alternate realization of Fig. 10.2-1(a).

Digital-to-Analog Converter

- Charge-scaling D/A Converter



$$v_{OUT} = [b_1 2^{-1} + b_2 2^{-2} + b_3 2^{-3} + \cdots + b_N 2^{-N}] V_{REF}$$

Digital-to-Analog Converter

- Combination of voltage-scaling and charge-scaling

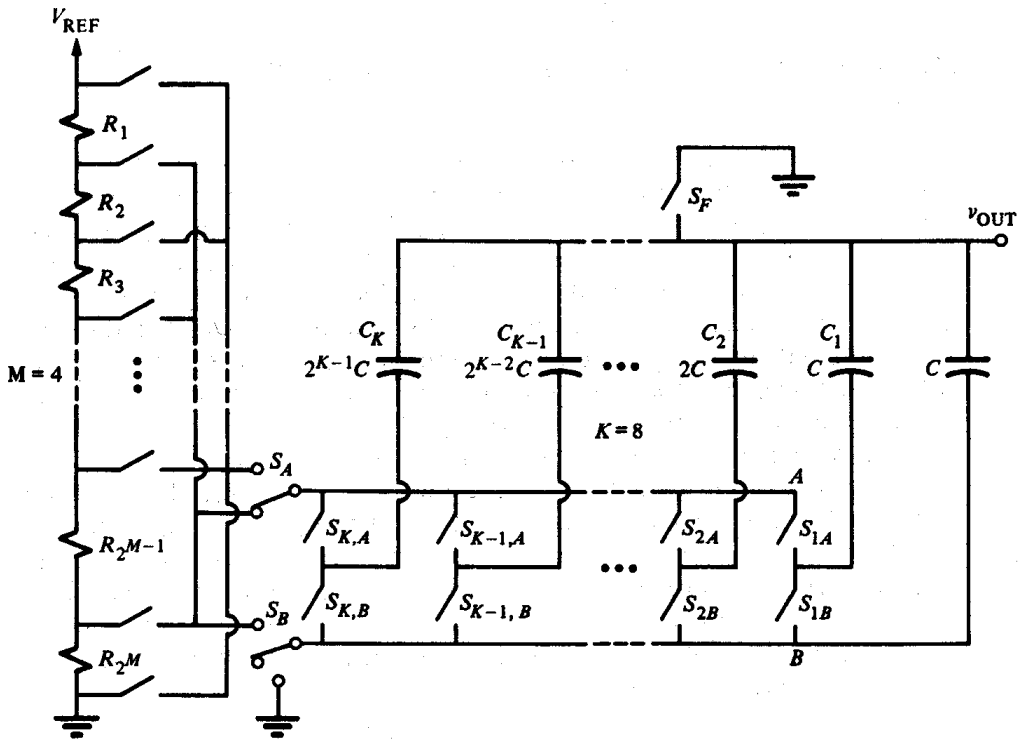
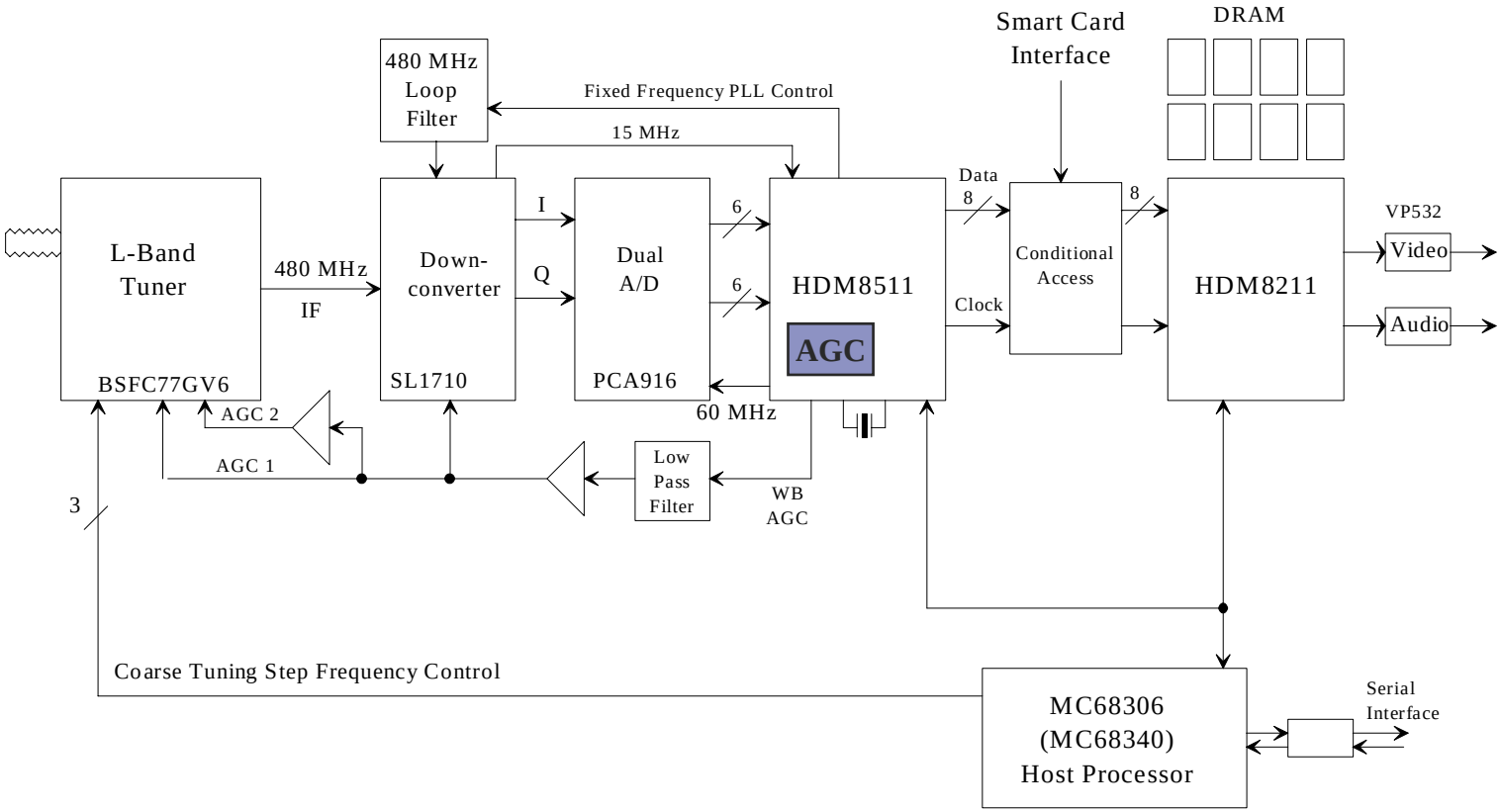


Figure 10.2-6 A D/A converter using a combination of voltage-scaling and charge-scaling techniques. The 4 MSBs are accomplished by voltage-scaling and the 8 LSBs are accomplished by charge-scaling.

Automatic-Gain-Control (AGC)

- Typical Set-Top Box Demodulator



A/D Converter Example

ANALOG DEVICES

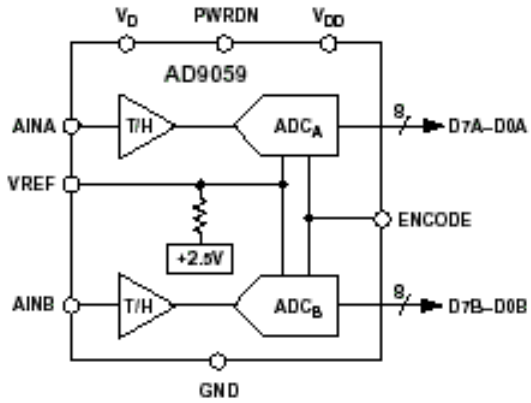
Dual 8-Bit, 60 MSPS A/D Converter

AD9059

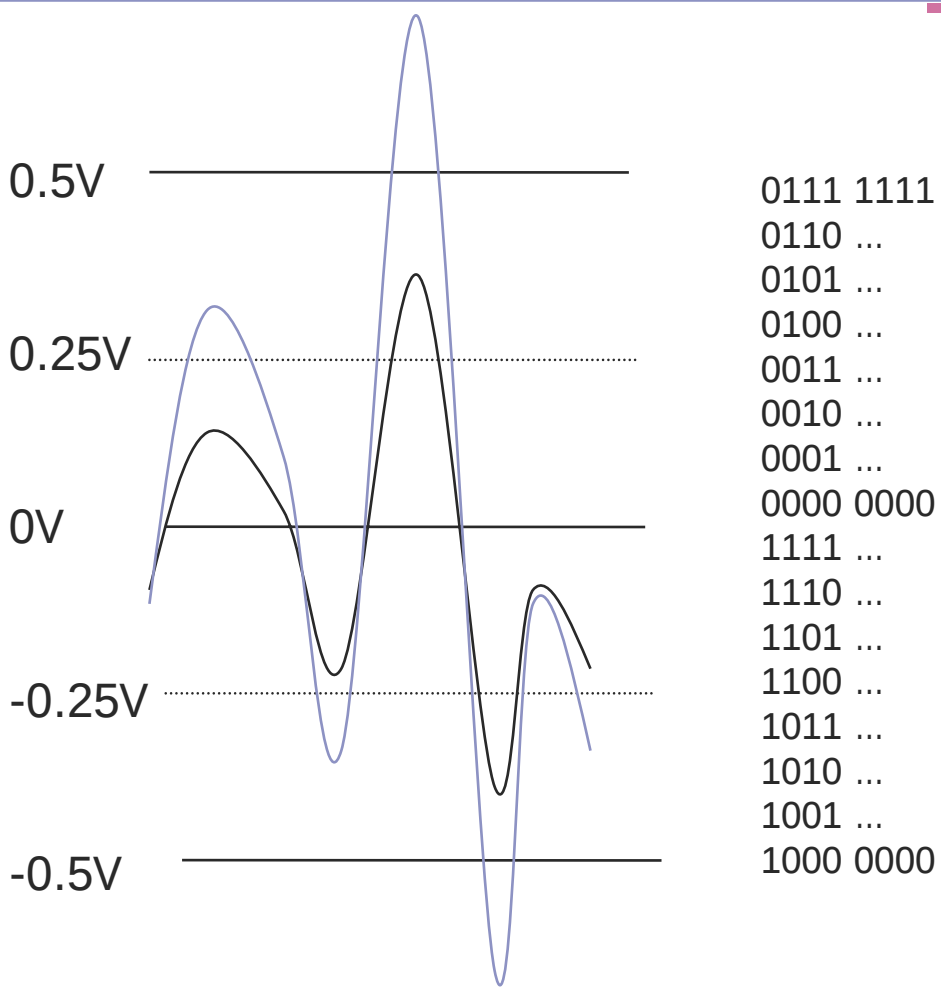
FEATURES
Dual 8-Bit ADCs on a Single Chip
Low Power: 400 mW Typical
On-Chip +2.5 V Reference and T/Hs
1 V p-p Analog Input Range
Single +5 V Supply Operation
+5 V or +3 V Logic Interface
120 MHz Analog Bandwidth
Power-Down Mode: < 12 mW

APPLICATIONS
Digital Communications (QAM Demodulators)
RGB & YC/Composite Video Processing
Digital Data Storage Read Channels
Medical Imaging

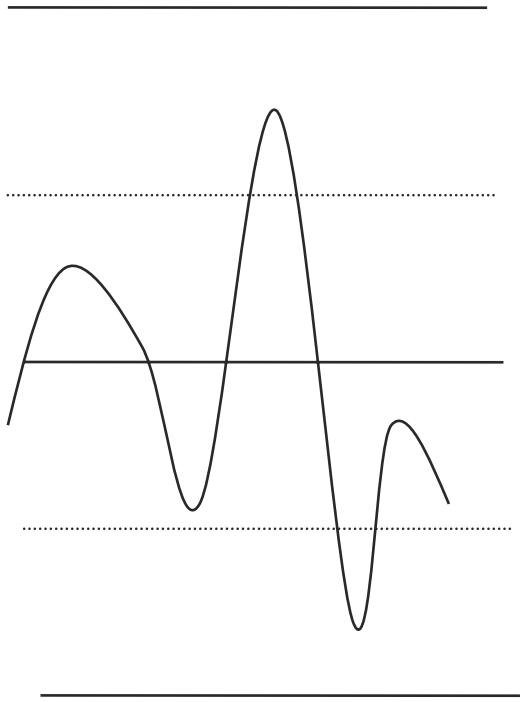
FUNCTIONAL BLOCK DIAGRAM



Analog-to-Digital Mapping

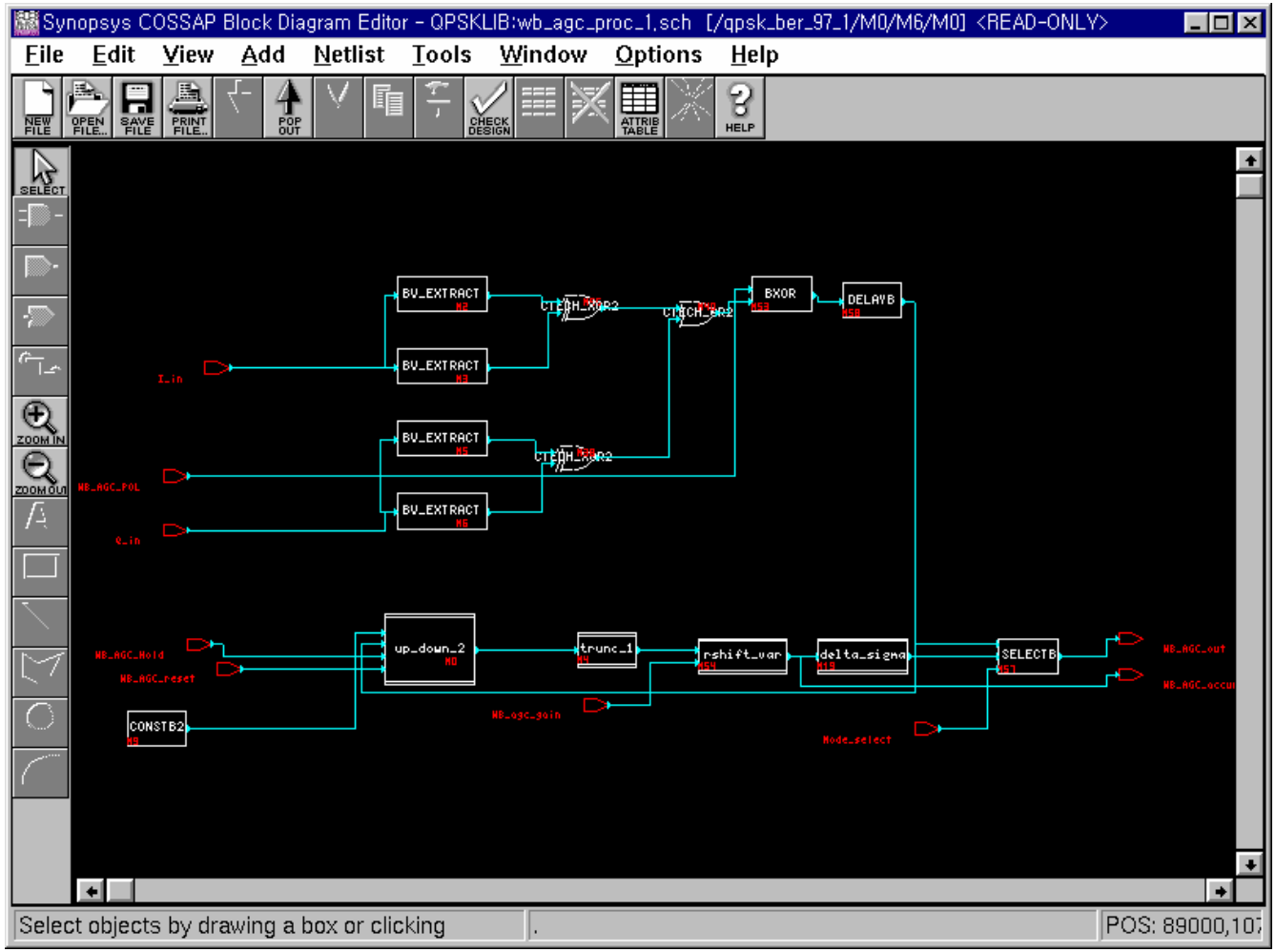


AGC Example



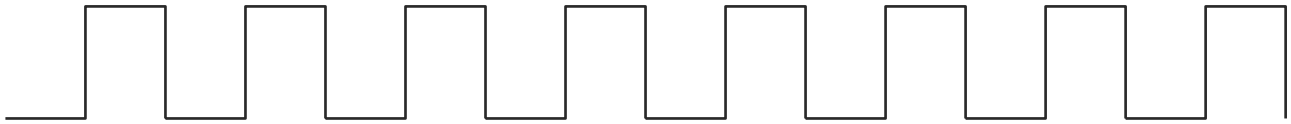
01 11
01 10
01 01
01 00
00 11
00 10
00 01
00 00
11 11
11 10
11 01
11 00
10 11
10 10
10 01
10 00

AGC Control Block

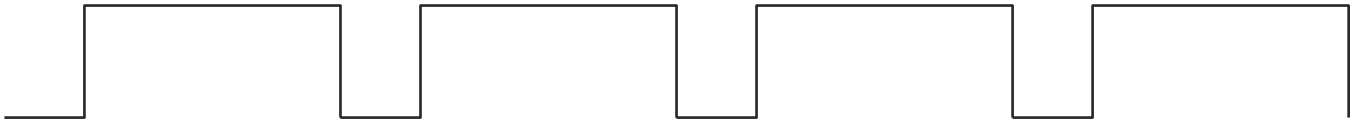


Output from AGC (PWM)

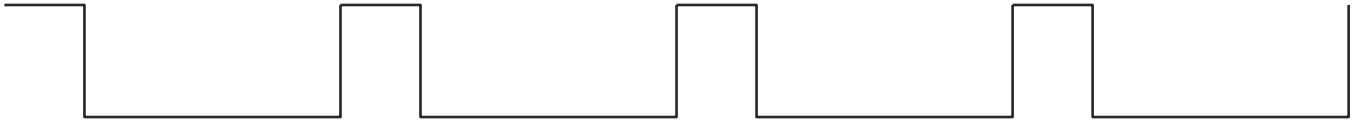
- Signal Adequate



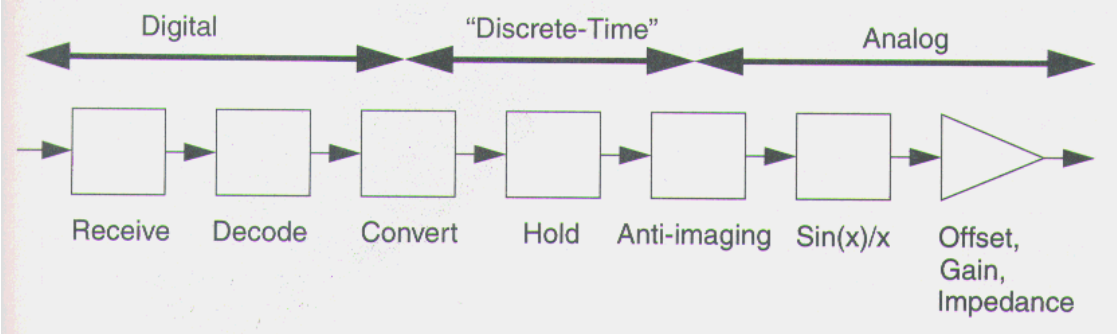
- Signal too Small

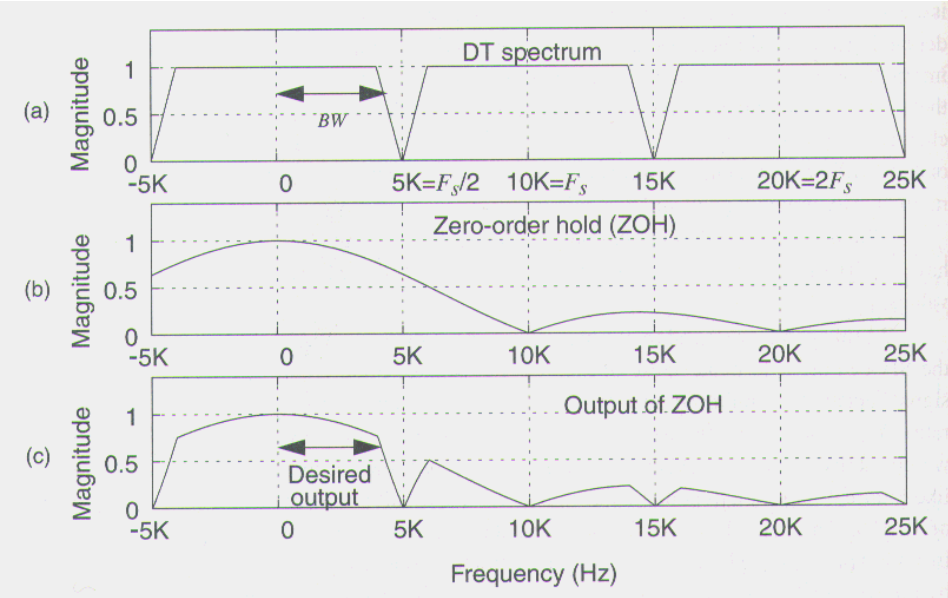


- Signal too Large



Digital-to-Analog Converter





Digital-to-Analog Converter

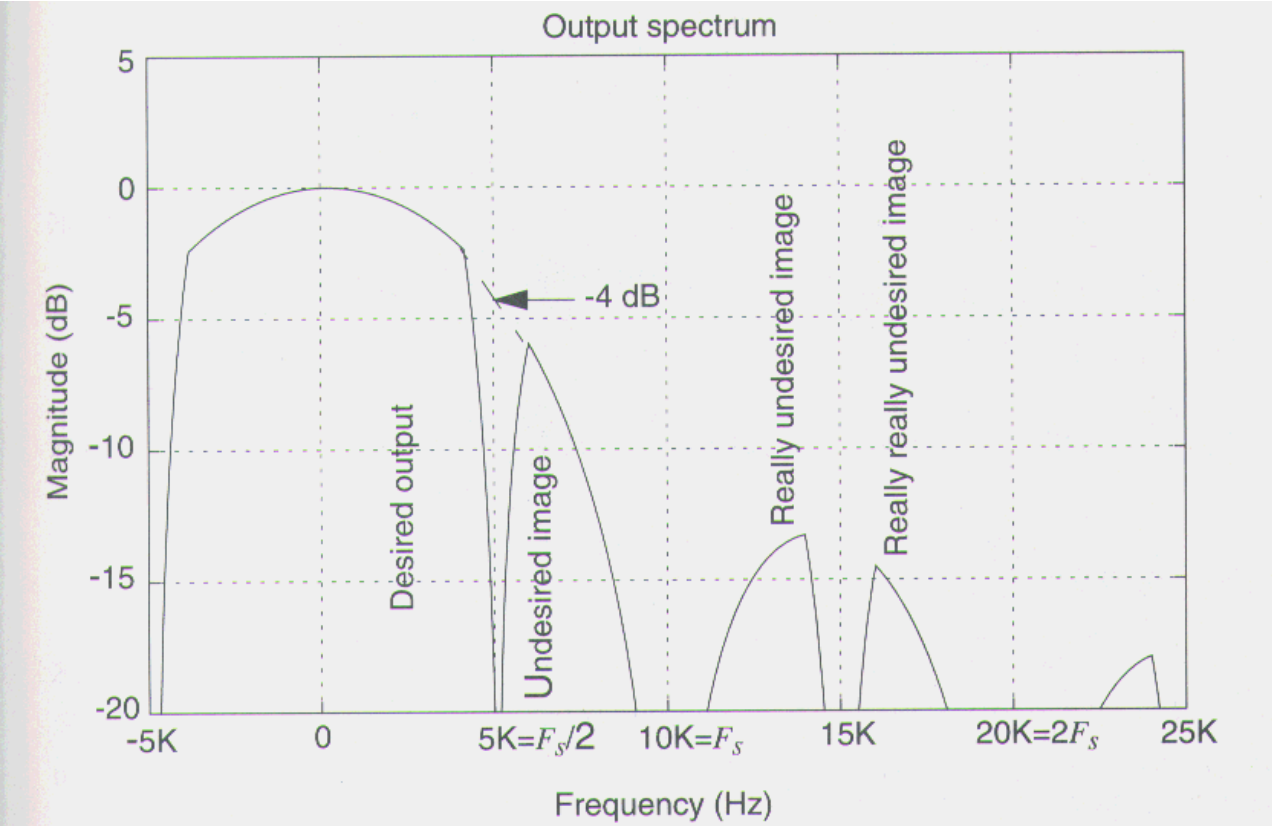
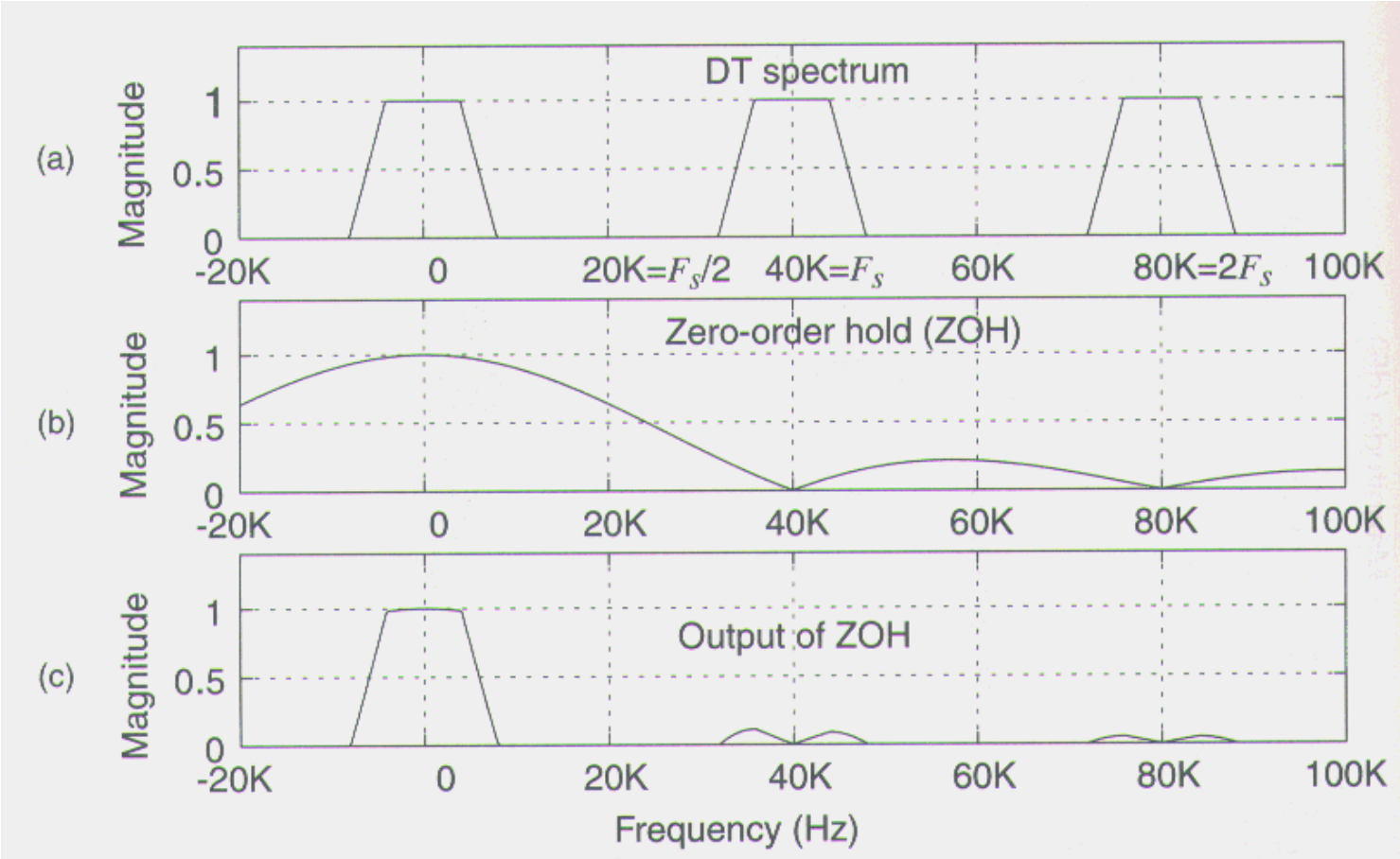


Figure 4-30 Expanded view of Figure 4-29c.

Sinx/x Compensation

- Compensation Filter Included in DAC
  - inverse sinc ($sinc^{-1}$) filter
  - x/sin(x)
- Pre-compensating Digital Filter Prior to DAC
- Stand-Alone Analog Filter after DAC
- Increase the Sampling Rate

Up-Sampling before DAC



Up-Sampling before DAC

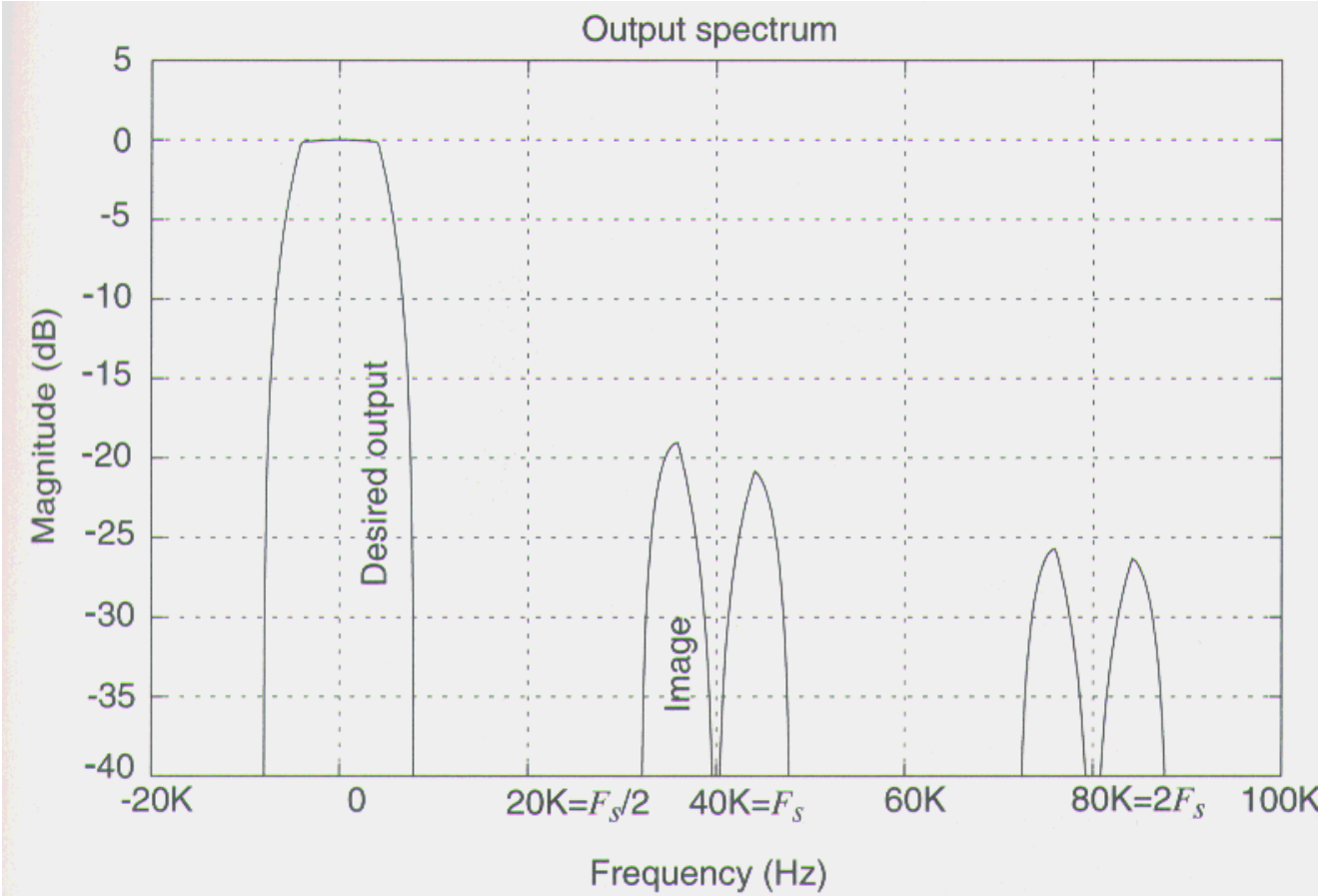


Figure 4-32 Expanded view of Figure 4-31c.

DSP Integrated Circuits

Chapter 7: Development Tools

Prof. Jaeseok Kim
Communication SoC Design Lab.
Yonsei University

Development Tools

- Software tools
  - Assemblers, linkers, simulators, debuggers, compilers, code libraries and real-time operating systems
- Hardware tools
  - Development boards and emulators
- Higher-level tools
  - Block-diagram based code generation environments

Software tools(1)

- Assemblers
  - Assemblers translate assembly-language mnemonics into binary object code
  - Assembler statements must be adequate to
    - Specify the structure of programs
    - Declare variables
    - Identify where referenced variables and I/O ports will reside the target
- Linkers
  - Linkers couples together relocatable object modules prepared by the assembler, including library subroutines
  - Produces absolute memory images for run-time execution

Software tools(2)

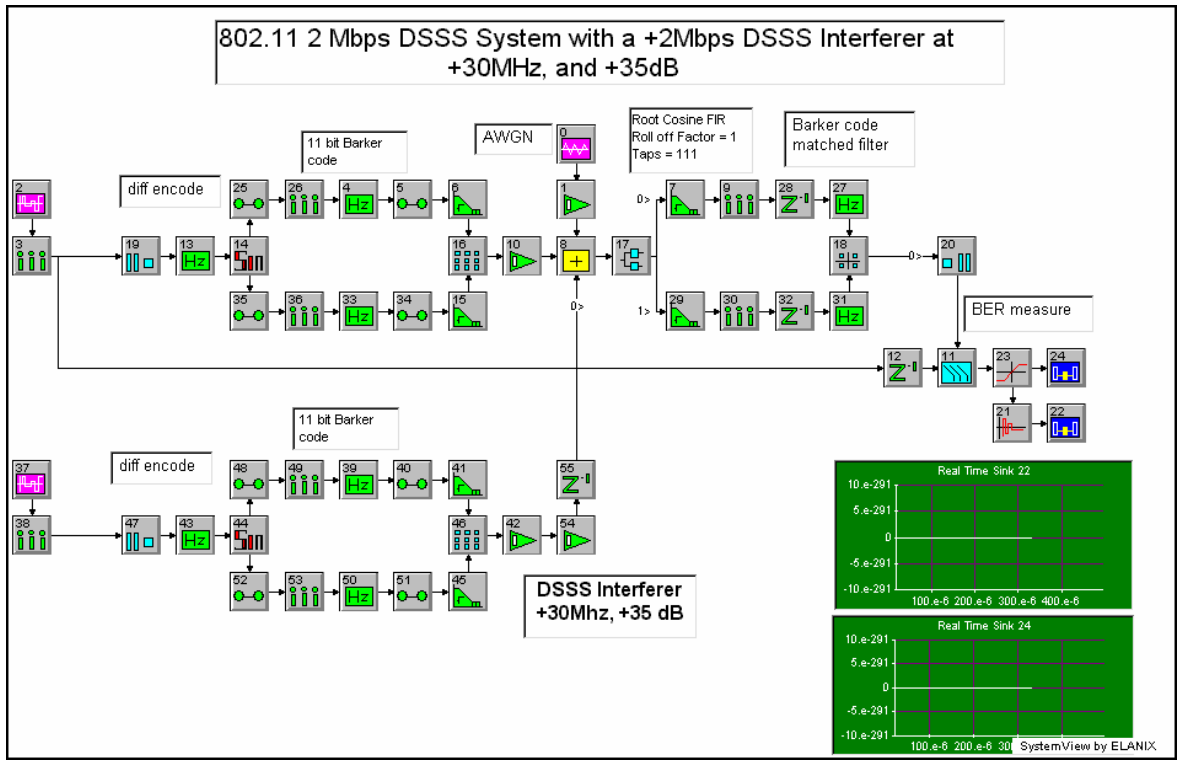
- Simulators
  - The simulator permits programs to be fine-tuned in software prior to making the significant effort to bring up the target system in hardware
- Debuggers
  - Front-end program that provides the user interface for an emulator or simulator
  - Key features
    - Data display and editing
    - Symbolic and source-level debugging
    - Single-stepping
    - Breakpoints

Hardware tools

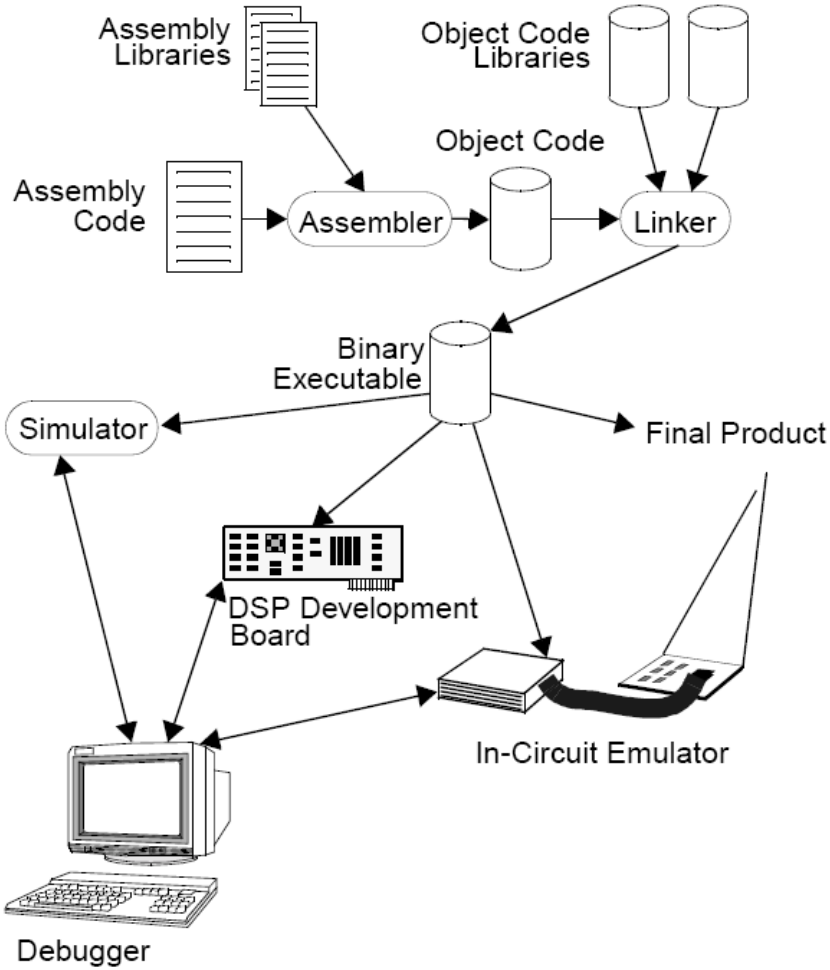
- The PROM splitter
  - The PROM splitter performs the function for target physical memory chips
  - Chunks of code destined for ROM are divided up into sizes that fit into regions of physical memory chips
- Target emulation
  - Emulator: A ream-time debug tool that takes over where the simulator leaves off
  - External hardware and software allow monitoring and controlling of the processor in the target system

Higher-level tools

- Block-diagram based code generation environments
  - SPW, Matlab simulllink, Systemview



Design flow


Assembly Libraries
Object Code Libraries
Object Code
Assembly Code
Assembler
Linker
Binary Executable
Simulator
Final Product
DSP Development Board
In-Circuit Emulator
Debugger